

```

    * @return mixed
    */
    public function handle($request, Closure $next)
    {
        if ($request->age <= 200) {
            return redirect('home');
        }

        return $next($request);    }
}

```

So the action that will take place here is if the `age` is less than or equal to 200, the user will be redirected to home page otherwise, the request will be passed further into the application.

It's interesting to note here that not all middleware happens after the request. Sometimes, it can run before the request is sent to the application. Also, there might be some middleware function you might want to run during every HTTP request to your application, list the middleware class in the `$middleware` property of your `app/Http/Kernel.php` class. This is also known as the global middleware. Alternatively, you might want to assign it to a specific route. So first, assign the middleware a key in your `app/Http/Kernel.php` file. The `$routeMiddleware` property of this class contains entries for the middleware included with Laravel. To add your own, append it to this list and assign it a key of your choosing. For example:

```

protected $routeMiddleware = [
    'auth' => \Illuminate\Auth\Middleware\
Authenticate::class,
    'auth.basic' => \Illuminate\Auth\Middleware\Authen
ticateWithBasicAuth::class,
    'bindings' => \Illuminate\Routing\Middleware\
SubstituteBindings::class,
    'can' => \Illuminate\Auth\Middleware\Authorize::class,
    'guest' => \App\Http\Middleware\
RedirectIfAuthenticated::class,
    'throttle' => \Illuminate\Routing\Middleware\
ThrottleRequests::class,
];

```

Then use the middleware method to assign middleware to a route:

```

Route::get('admin/profile', function () {
    //
})->middleware('auth');

```